

Functional Programming in F# from Zero to Hero

Internal Engineering Learning Series

A decorative horizontal bar at the bottom of the slide, consisting of two overlapping blue bands of different shades.

Series Goal

Open to Engineering team members interested in Functional Programming and F#.

1. Introduce Functional Programming concepts using F# and .NET.
 2. Start with fundamentals, then progress toward practical and advanced topics.
 3. Combine theory, examples, demos, and hands-on exercises.
 4. Help attendees write code that is easier to reason about, test, compose, and maintain.
- 
- A decorative horizontal bar at the bottom of the slide, consisting of three overlapping blue rectangles of varying shades, creating a gradient effect.

Progressive Learning Path

From introductory concepts to advanced F# techniques.

1. Functional Programming mindset and F# basics
 2. Functions, immutability, type inference, records, and tuples
 3. Higher-order functions, partial application, pipelining, and composition
 4. Options, pattern matching, discriminated unions, and safer domain modeling
 5. Collections, units of measure, object-oriented interop, recursion, async, and agents
- 

Sessions plan

From introductory concepts to advanced F# techniques.

1. Introduction to Functional Programming and F#
2. F# Language Basics
3. Functions as First-Class Values
4. Pipelining and Composition
5. Modeling Data with F# Types
6. Options, Pattern Matching, and Null Safety
7. Functional Collections
8. Units of Measure and Safer Business Logic
9. Object-Oriented Interop in F#
10. Recursion and Active Patterns
11. Implement a DSL in F#
12. Asynchronous Programming in F#
13. Agents and Concurrent Programming

Session 1

INTRODUCTION TO FUNCTIONAL PROGRAMMING AND F#

A decorative horizontal bar at the bottom of the slide, consisting of three overlapping blue bands of varying shades, creating a gradient effect.

Session 1 Plan

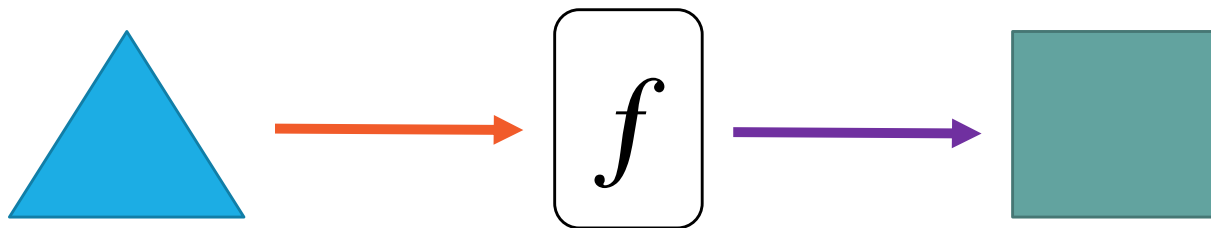
Introduction to Functional Programming and F#

1. Why Functional Programming matters.
2. Compare imperative and functional styles.
3. Immutability, pure functions, expressions, and side effects.
4. F# is a functional-first language on .NET.

Walkaway

Functional Programming is a mindset for making code more predictable by reducing hidden state and composing explicit transformations.

What Is Functional Programming?

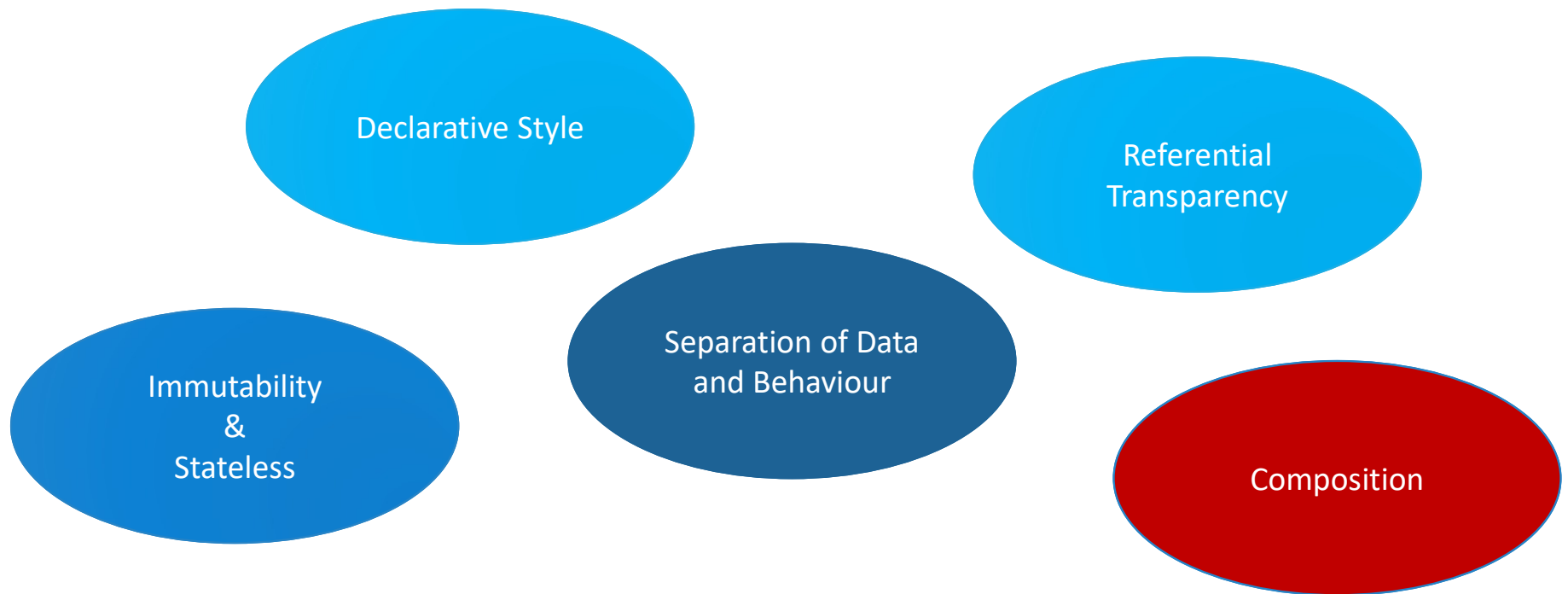


Referential transparency:
same input – same result



Information about possible inputs and
outcomes

Functional Core Concepts



Why Functional Programming?



Composable



Easy to reason about



Easier to unit test



Reducing
complexity

What Is Functional Programming?

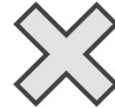
```
public double Calculate(double x, double y)
{
    return x * x + y * y;
}
```



Same input – same result

```
public long TicksElapsedFrom(int year)
{
    DateTime now = DateTime.Now;
    DateTime then = new DateTime(year, 1, 1);

    return (now - then).Ticks;
}
```



Result is always different

A different mindset

Imperative mindset

- Data + procedures.
- Mutable variables and data structures.
- Relies on side effects to do things.
- Return type is usually void.

Purely functional mindset

- Data + (pure) functions.
- Immutable variables and data structures.
- Functions do not perform side effects.
- Functions are defined through their type signature.



F# Language

F# is compatible: runs on .NET Core (cross-platform)

F# is simple and compact: a small language core with a flat learning curve

F# is high-level: Higher level of abstraction by combining OO with FP

F# is statically typed: strong type inference and type system

Easy to parallelize: Sets you up for multi-core programming

Easy to test and debug

Increase of productivity

A large white Greek letter lambda (λ) is centered on a solid blue square background.

Make the code predictable

a -> b

Make the code predictable

```
// create two customers
```

```
var cust1 = new Customer(99, "J Smith");
```

```
var cust2 = new Customer(99, "J Smith");
```

```
// true or false?
```

```
cust1 == cust2;
```

You cannot tell!

Not predictable!

Make the code predictable

```
// create two customers
```

```
var cust = new Customer(99, "J Smith");
```

```
var order = new Order(10, "Pizza");
```

```
// true or false?
```

```
Cust.Equals(order);
```

This should be a bug!

Why are you attempting to compare
an order to a customer?

Make the code predictable

```
// create a customer
var cust = new Customer(99, "J Smith");

// add it to a set
var processedCustomers = new HashSet<Customer>();
processedCustomers.Add(cust);

// process it
ProcessCustomer(cust);
// Is the customer still in the set? True or false?
processedCustomers.Contains(cust);
```

You cannot tell!

If ProcessCustomer mutates the customer, it might change the hash

Make the code predictable

```
// create a customer
var cust = new ImmutableCustomer(99, "J Smith");

// add it to a set
var processedCustomers = new HashSet<ImmutableCustomer>();
processedCustomers.Add(cust);

// process it
var changedCustomer = ProcessCustomer(cust);
// Is the customer still in the set? True or false?
processedCustomers.Contains(cust);
```

Immutability forces changed values to be returned explicitly. Original value unchanged.

Immutability

F#

```
F# Interactive for F#  
> let x = 5;;
```

Immutability

F#

```
F# Interactive for F#  
> let x = 5;;  
val x : int = 5  
>
```

Immutability

F#

F# Interactive for F#

```
> let x = 5;;
```

```
val x : int = 5
```

```
> x = x + 1;;
```

Immutability

F#

```
F# Interactive for F#  
> let x = 5;;  
val x : int = 5  
> x = x + 1;;  
val it : bool = false  
>
```

Immutability

F#

F# Interactive for F# 3.1

```
> let x = 5;;  
val x : int = 5  
> x = x + 1;;  
val it : bool = false  
> x <- 7;;
```

Immutability

F# Interactive for F#

```
> let x = 5;;
```

```
val x : int = 5
```

```
> x = x + 1;;
```

```
val it : bool = false
```

```
> x <- 7;;
```

```
  x <- 7;;
```

```
^^^^^^
```

```
//stdin(19,1): error FS0027: This  
value is not mutable
```

```
>
```

F#

CompSci 101

$X = X + 1$

Increment!

High school math

$X = X + 1$

You fail!

Why Does Immutability Matter?



Increased readability



A single place for validating invariants
Reason vs understanding the code



Automatic thread safety

F# Semantics: Session 1

Introduction to Functional Programming and F#

1. let bindings and immutable values by default.
2. Expressions return values.
3. Pure functions depend only on their inputs.
4. Side effects include console output, file I/O, databases, network calls, and mutation.

Bindings

let x = 1

~~x = x + 1~~

let y = x + 1

let mutable x = 1
x <- 2

Demo 1

BINDINGS | FUNCTIONS | TUPLES | RECORDS

